

# LAB

## Topic 2: Simple SQL Queries



Lecturer: Lê Thị Nhàn, PhD.  
Lab instructors: Nguyễn Ngọc Toàn, MSc.  
Nguyễn Ngọc Minh Châu, MSc.

# 1. Basic SQL syntax

## a) SQL Syntax Structure

```
SELECT [ALL | DISTINCT | TOP(n) ] <attributes_list_1>
FROM <table_1, table_2, table_3>
[WHERE <search_condition_1>]
[GROUP BY <attributes_list_2>]
[HAVING <search_condition_2>]
[ORDER BY <attribute_1 ASC|DESC, attribute_2 ASC|DESC, ...>]
```

### Note:

- SQL conditions can be simple comparisons: >, <, = (**not ==**), >=, <=, <> (**not !=**) or check **IS NULL** or **IS NOT NULL** or set operation **IN**, **NOT IN**, **EXISTS**, **NOT EXISTS** or pattern matching **LIKE**. SQL conditions can be combined by logical operators **AND**, **OR**, and **NOT**.
- SQL keywords are case insensitive!

## b) Example

### Retrieve all data from a table

```
SELECT *
FROM <table_name>
```

Example:

```
SELECT *
FROM Instructor
```

### Retrieve some specific columns

- Retrieve (instructor\_id, instructor\_name) of Instructor

```
SELECT instructor_id, instructor_name
FROM Instructor
```

### Retrieve data with condition in WHERE clause

- Retrieve (Student\_name, gender) of Students belonging to class 24CNTT1.

```
SELECT Student_name, gender
FROM Student
WHERE class = '24CNTT1'
```

- Retrieve (Student\_name, Student\_id) of the female Students belonging to class 24CNTT1

```
SELECT Student_name, Student_id
FROM Student
WHERE gender = 'F' and class = '24CNTT1'
```

- Retrieve the Students whose name start with letter 'H'

```
SELECT *
FROM Student
WHERE Student_name LIKE 'H%'
```

Note: The percent sign '%' represents zero, one, or multiple characters

- Retrieve the Students belonging to Department CS or Department SE

```
SELECT *
FROM Student
WHERE Department_id IN ('CS', 'SE')
```

OR:

```
SELECT *
FROM Student
WHERE Department_id = 'CS' OR Department_id = 'SE'
```

- Retrieve the Instructor having no phone

```
SELECT *
FROM Instructor
WHERE phone IS NULL
```

- Retrieve the instructor\_id and instructor\_name of Instructor who have taught in the role of "lecturer."

```
SELECT DISTINCT I.instructor_id, I.instructor_name
FROM Instructor I
JOIN Teaching T ON I.instructor_id = T.instructor_id
WHERE T.teaching_role = 'Lecturer'
```

- **Some datetime functions:**

```
year: get the year value of a datetime input
month: get the month value of a datetime input
day: get the day value of a datetime input
datepart: get part values of datetime value
```

**datediff**: calculate the distance (by day, month or year, ...) of 2 **datetime** inputs

**getdate**: return the current date

- Retrieve the Students born in 2000

```
SELECT *  
FROM Student  
Where year(birthdate) = 2000
```

- Retrieve (Student\_name, Age) of the Students belonging in class 24CNTT1. The result should be sorted in ascending order by their age

```
SELECT Student_name, datediff(yyyy, birthdate, getdate()) as Age  
FROM Student  
WHERE class = '24CNTT1'  
ORDER BY Age
```

## SQL UNION, INTERSECT, EXCEPT

- The **SQL UNION** operator combines the results of two **SELECT** statements which have the same

columns and same data-type in each column. **UNION** operator will return all different values.

However, if you want to list all values, using **UNION ALL** instead of **UNION**.

- **EXCEPT** return rows from data set 1 what doesn't exists in data set 2 and eliminates duplicates rows
- **INTERSECT** returns rows that appear in both data sets and eliminate duplicate rows

### Example:

- Retrieve instructor\_name of Instructor who have a salary greater than 2000 or Instructor with the name 'John'

```
SELECT instructor_name FROM instructor WHERE salary >2000  
UNION - - UNION ALL  
SELECT instructor_name FROM instructor WHERE name LIKE '%John%'
```

Note: **UNION ALL** keeps all duplicate rows, while **UNION** removes them.

- Retrieve instructor\_id of Instructor who have not participated in teaching in the role of "lecturer".

```
SELECT instructor_id FROM instructor  
EXCEPT  
Select instructor_id FROM Teaching WHERE teaching_role= 'Lecturer'
```

## SQL JOIN

- Join condition in **WHERE** clause:

```
SELECT *  
FROM Student, Department  
WHERE Student.Department_id = Department.Department_id
```

OR

```
SELECT *  
FROM Student s, Department d -- alias  
WHERE s.Department_id = d.Department_id
```

**Note:** If a SQL join condition (**s.Department\_id = d.Department\_id**) is omitted, the join operation will result in a Cartesian product. The Cartesian product returns a number of rows equal to the product of all rows in all the joined tables. For example, if the first table has 20 rows and the second table has 10 rows, the result will be 20 \* 10 or 200 rows

### JOIN operators (Inner JOIN, Left JOIN, Right JOIN):

- Inner Join

```
SELECT *  
FROM Student s INNER JOIN Department d ON s.Department_id = d.Department_id
```

- Left Join

```
SELECT *  
FROM Student s LEFT JOIN Department d ON s.Department_id = d.Department_id
```

- Right Join

```
SELECT *  
FROM Student s RIGHT JOIN Department d ON s.Department_id = d.Department_id
```

# GROUPING ROWS IN A TABLE

## a) Syntax Structure

```
SELECT [ALL|DISTINCT|TOP(n)] <column_1, column_2,..., aggregate_function
(column_X)>
FROM <table_1, table_2, table_3>
[WHERE <conditions on each row>]
GROUP BY <column_1, column_2,...> -- rows will be grouped based on the value of
col1, then col2,...
[HAVING <conditions on each group>]
[ORDER BY <attribute_1 ASC|DESC, attribute_2 ASC|DESC, ...>]
```

### Note:

- **GROUP BY** statement is often used in combination with aggregate functions (**MIN**, **MAX**, **SUM**, **COUNT**, **AVG**) to perform calculations on the grouped data:
  - + The aggregation functions `min(salary)`, `max(salary)`, `avg(salary)`, and `sum(salary)` allow retrieving the minimum, maximum, average, and sum of the salary column, respectively.
  - + The aggregation functions `count(student_id)`, `count(*)`, and `count(distinct student_id)` count the number of `student_id` values, the number of rows, and the number of `student_id` values without duplication.
- A column used in statements processed after the **GROUP BY** statement such as **SELECT**, **HAVING** must be appeared in the **GROUP BY** statement or embedded in an aggregate function.

## b) Example

### Basic aggregation with (implicit) grouping

- Retrieve the maximum and minimum salary of instructors in the Computer Science department.

```
SELECT max(salary) as MaxSalary, min(salary) as MinSalary
FROM Instructor I join Department D on I.department_id = D.department_id
WHERE D.department_name = 'Computer Science'
```

- Retrieve the number of courses managed by the Information System department.

```
SELECT count(*) as nb_course
--or count(distinct course_id), count(course_id)
FROM Course C join Department D on C.department_id = D.department_id
WHERE D.department_name = 'Information System'
```

## Aggregation and (explicit) grouping

- Retrieve the ID and name of departments along with the average salary of instructors in each department. Order departments by department\_head in descending order.

```
SELECT D.department_id, D.department_name, avg(salary) as AvgSalary
FROM Instructor I join Department D on I.department_id = D.department_id
GROUP BY D.department_id, D.department_name, D.department_head
--department_name and department_head must be included in the GROUP BY
--statement to be used in the ORDER BY and SELECT statements.
ORDER BY D.department_head DESC
```

- Retrieve the department name and the average salary of instructors in those departments where the number of instructors is more than 2.

```
SELECT D.department_name, avg(salary) as AvgSalary
FROM Instructor I join Department D on I.department_id = D.department_id
GROUP BY D.department_id, D.department_name
HAVING count(I.instructor_id) > 2
```

- Retrieve the department id, the gender, and the number of students for each gender in each department.

```
SELECT D.department_id, S.gender, count(student_id) as nb_student
FROM Department D join Student S on D.department_id = S.department_id
GROUP BY D.department_id, S.gender
/* Students will be grouped by department_id values, and then the students in
the same department will be further divided by their gender.*/
```

## Aggregation with null values

- Retrieve the ID and name of ALL departments along with the number of instructors in each department.

```
SELECT D.department_id, D.department_name,  
       ISNULL(count(I.instructor_id),0) as nb_ins  
FROM Instructor I right join Department D on I.department_id = D.department_id  
GROUP BY D.department_id, D.department_name
```

## NESTED QUERIES

A SQL subquery (inner or nested query) can be used as a parameter of a condition in a statement (e.g., **SELECT**, **FROM**, **WHERE**) of another SQL query (outer query). Several comparisons can be used to connect inner and outer queries: = **ANY** (or = **SOME**). Other operators **>**, **<**, **>=**, **<=**, **<>** can be combined with **ANY** or **SOME** or **ALL**. The **IN/NOT IN** and **EXISTS/NOT EXISTS** can also be used in nested queries.

### 1. Nested queries with comparison operators

- Retrieve the instructor\_id and instructor\_name of the instructor(s) with the highest salary

```
SELECT instructor_id, instructor_name  
FROM Instructor  
WHERE salary = (SELECT MAX(salary)  
                FROM Instructor)
```

### 2. Nested queries with ALL, SOME/ANY

- The another solution:

```
SELECT instructor_id, instructor_name  
FROM Instructor  
WHERE salary >= ALL (SELECT salary  
                     FROM Instructor)
```

### 3. Correlated nested queries

A subquery referencing at least one attribute from the outer query is called a correlated nested query.

- Retrieve the ID and name of instructors who have the highest salary in each department.

```
SELECT I1.instructor_id, I1.instructor_name
FROM Instructor I1
WHERE I1.salary >= ALL (SELECT I2.salary
                        FROM Instructor I2
                        WHERE I2.department_id = I1.department_id)
```

### 4. Nested queries with IN / NOT IN

- Retrieve the ID and name of instructors in the Software Engineering department who have never worked as a TA (i.e., teaching assistant).

```
SELECT I.instructor_id, I.instructor_name
FROM Instructor I join Department D on I.department_id = D.department_id
WHERE D.department_name = 'Software Engineering' AND I.instructor_id NOT IN
      (SELECT T.instructor_id FROM Teaching T WHERE T.teaching_role = 'TA')
```

- Retrieve the ID and name of instructors who teach in at least one section with John.

```
SELECT I.instructor_id, I.instructor_name
FROM Instructor I join Teaching T on I.instructor_id = T.instructor_id
WHERE T.section_id IN (SELECT T2.section_id
                       FROM Instructor I2 join Teaching T2
                       on I2.instructor_id = T2.instructor_id
                       WHERE I2.instructor_name LIKE '% John %')
```

### 5. Nested queries with EXISTS / NOT EXISTS

- Retrieve the ID and name of instructors who have the highest salary.

```
SELECT I1.instructor_id, I1.instructor_name
FROM Instructor I1
WHERE NOT EXISTS (SELECT *
                  FROM Instructor I2
                  WHERE I2.salary > I1.salary)
```

**6. Complex nested queries with grouping**

- Retrieve the ID and name of departments that have the highest average salary.

```
SELECT D.department_id, D.department_name
FROM Department D join Instructor I on D.department_id = I.department_id
GROUP BY D.department_id, D.department_name
HAVING AVG(I.salary) >= ALL (SELECT AVG(I2.salary)
                             FROM Instructor I2
                             GROUP BY I2.department_id)
```